Functions are a simple yet effective way to implement business logic and delegate infrastructural concerns to the framework. In the next section you'll learn how to bind functions to message channels on RabbitMQ using Spring Cloud Stream.

10.4 Processing messages with Spring Cloud Stream

The principles that drive the Spring Cloud Function framework can also be found in Spring Cloud Stream. The idea is that you, as a developer, are responsible for the business logic, while the framework handles infrastructural concerns like how to integrate a message broker.

Spring Cloud Stream is a framework for building scalable, event-driven, and streaming applications. It's built on top of Spring Integration, which offers the communication layer with message brokers; Spring Boot, which provides auto-configuration for the middleware integration; and Spring Cloud Function, which produces, processes, and consumes events. Spring Cloud Stream relies on the native features of each message broker, but it also provides an abstraction to ensure a seamless experience independently of the underlying middleware. For example, features like consumer groups and partitions (native in Apache Kafka) are not present in RabbitMQ, but you can still use them thanks to the framework providing them for you.

My favorite Spring Cloud Stream feature is that you can drop a dependency in a project like Dispatcher Service and get functions automatically bound to an external message broker. The best part of it? You don't have to change any code in the application, just the configuration in application.yml or application.properties. In previous versions of the framework, it was